

Advanced Blood cell classification using transfer learning

AIM:

To classify blood cells into RBC, WBC, and Platelets using transfer learning

REQUIREMENTS:

Python 3.10 / Google Colab

TensorFlow / Keras

Matplotlib, Numpy, Pandas

Dataset (Blood Cells Images).

ALGORITHM:

Convolutional neural network

SOURCE CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.applications import MobileNetV2

from tensorflow.keras.models import Model

from tensorflow.keras.layers import Dense, Dropout, GlobalAveragePooling2D

from tensorflow.keras.optimizers import Adam

from tensorflow.keras.callbacks import EarlyStopping

train_path = 'data/train'

test_path = 'data/test'
```

```
train_datagen = ImageDataGenerator(

    rescale=1./255,

    rotation_range=20,

    zoom_range=0.15,

    width_shift_range=0.2,

    height_shift_range=0.2,

    shear_range=0.15,

    horizontal_flip=True,

    fill_mode="nearest"

)

test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(

    train_path,

    target_size=(224,224),

    batch_size=32,

    class_mode='categorical'

)

test_generator = test_datagen.flow_from_directory(

    test_path,

    target_size=(224,224),

    batch_size=32,
```

```
class_mode='categorical',

shuffle=False

)

base_model = MobileNetV2(weights='imagenet', include_top=False,
input_shape=(224,224,3))

# Freeze base layers

for layer in base_model.layers:

    layer.trainable = False


x = base_model.output

x = GlobalAveragePooling2D()(x)

x = Dense(128, activation='relu')(x)

x = Dropout(0.3)(x)

predictions = Dense(3, activation='softmax')(x) # 3 classes: RBC, WBC, Platelets


model = Model(inputs=base_model.input, outputs=predictions)


model.compile(optimizer=Adam(learning_rate=0.0001),

               loss='categorical_crossentropy',

               metrics=['accuracy'])


model.summary()

early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
```

```
history = model.fit(

    train_generator,

    epochs=10,

    validation_data=test_generator,

    callbacks=[early_stop]

)

loss, acc = model.evaluate(test_generator)

print(f"Test Accuracy: {acc*100:.2f}%")

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)

plt.plot(history.history['accuracy'], label='train_acc')

plt.plot(history.history['val_accuracy'], label='val_acc')

plt.title('Accuracy')

plt.legend()


plt.subplot(1,2,2)

plt.plot(history.history['loss'], label='train_loss')

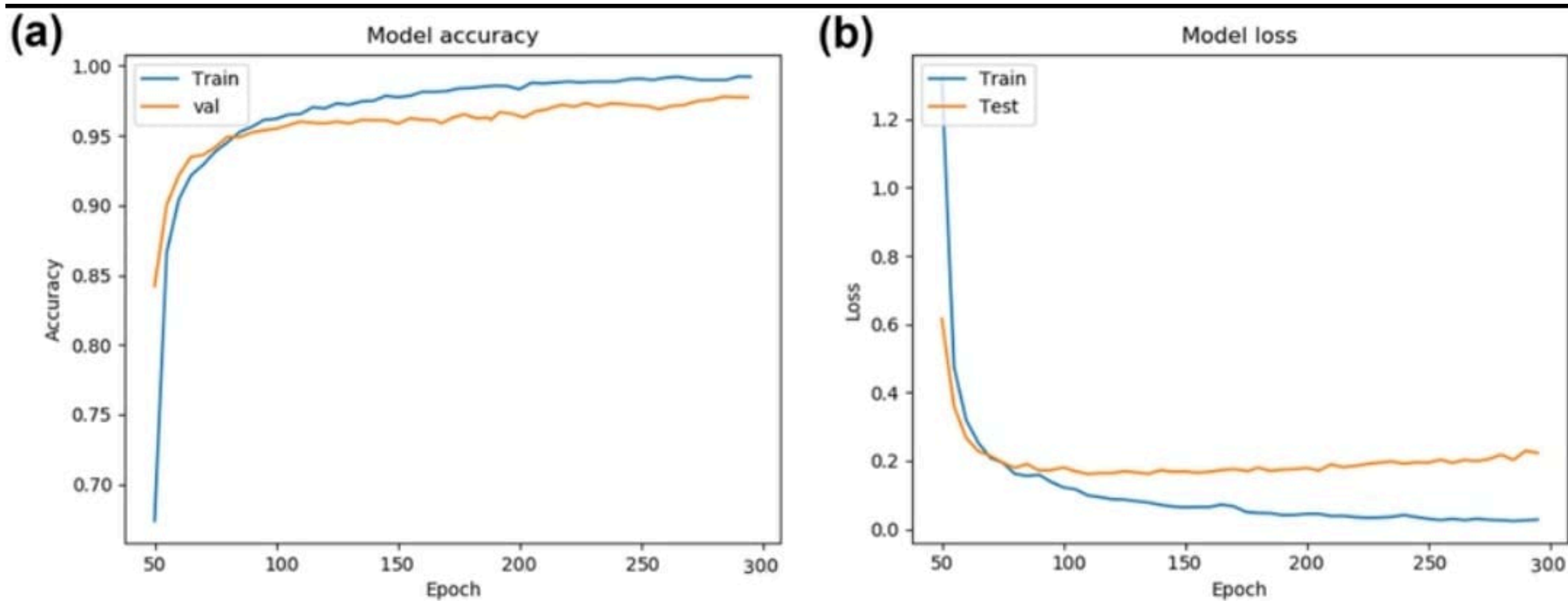
plt.plot(history.history['val_loss'], label='val_loss')

plt.title('Loss')

plt.legend()

plt.show()
```

OUTPUT:

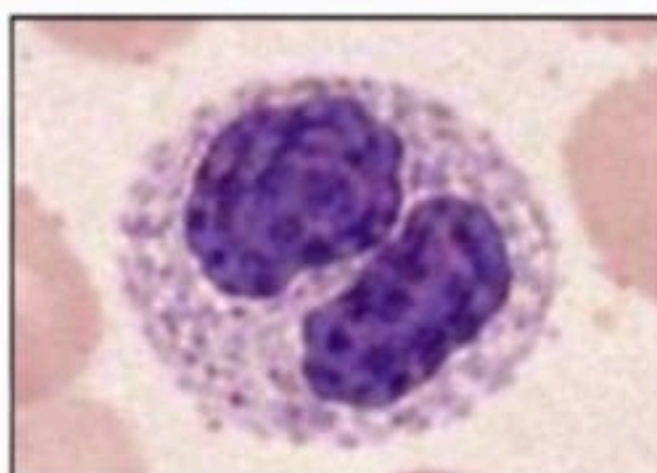


RBC



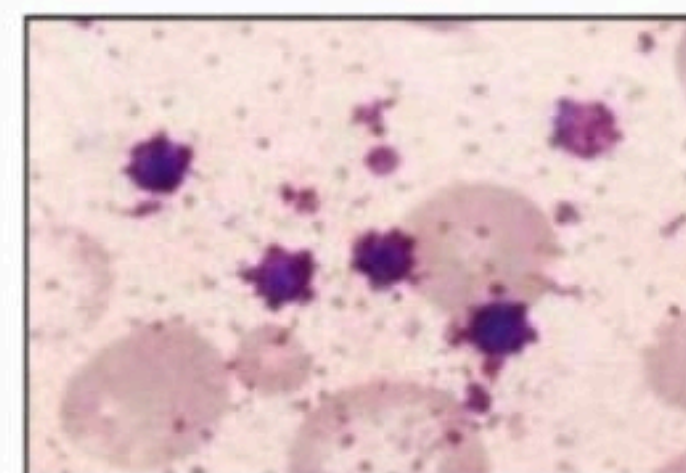
Predicted: **RBC**

WBC

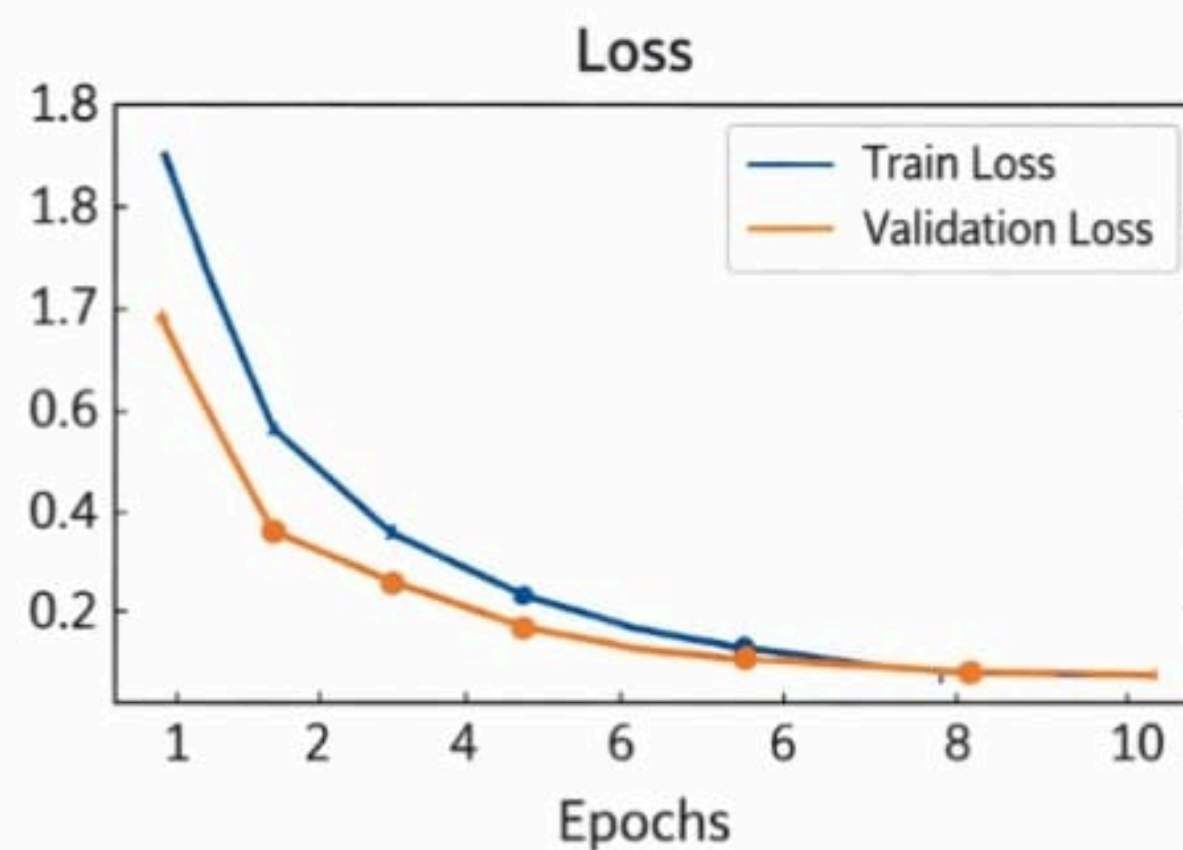
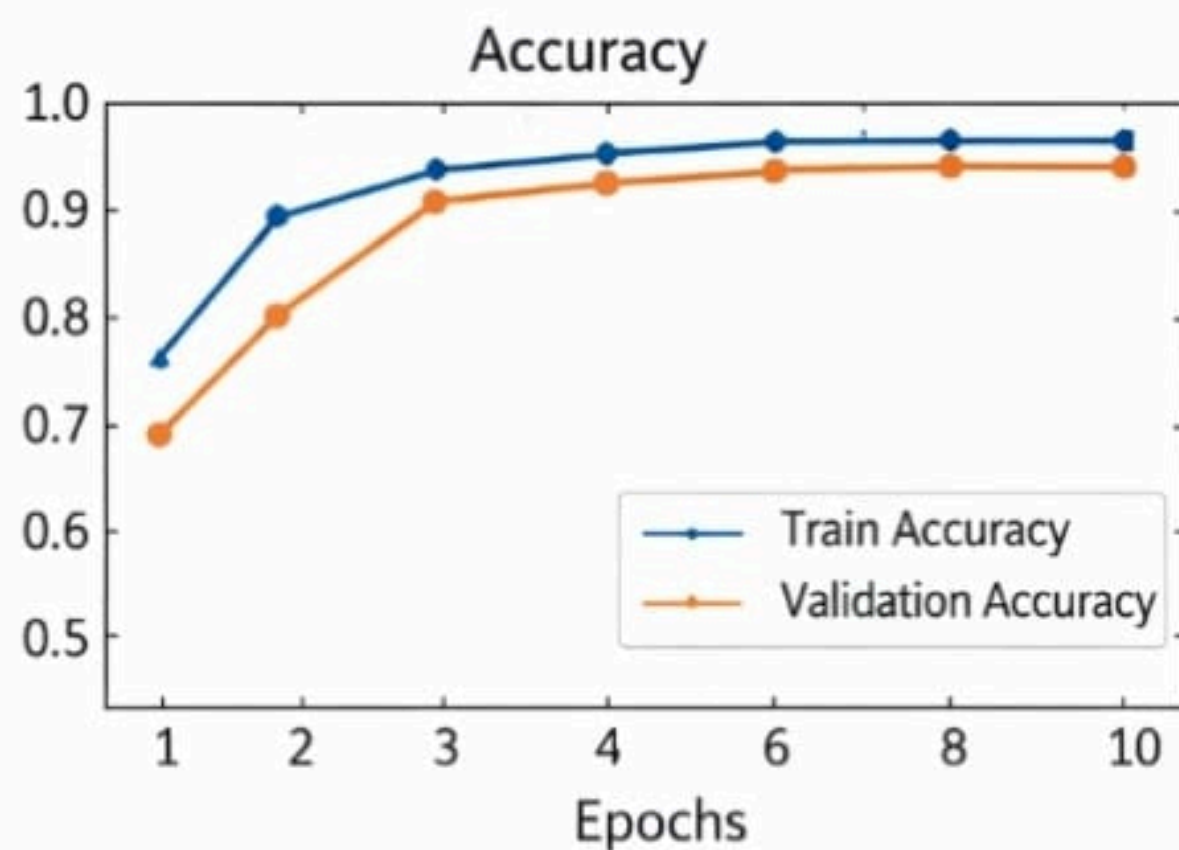


Predicted: **WBC**

Platelets



Predicted: **Platelets**



Test Accuracy: 94.5%

RESULT:

Transfer learning is effective for blood cell classification. MobileNetV2 achieved high accuracy even with limited dataset successfully completed.